

raft-kv: The Brief

A Distributed Key-Value Store with Raft Consensus,
Deterministic Simulation, and a Linearizability Checker

Explainer document – generated for Salman Adnan’s portfolio

This document is a concise, accurate companion to the full Deep Dive. Every fact below comes from the project’s source code and README; nothing is invented.

Contents

1	What and Why	1
2	Project Layout	2
3	The Raft Core	2
4	The Replicated State Machine	3
5	Two Transports, One Determinism Story	4
6	The Chaos Harness	4
7	The Linearizability Checker	4
8	Verification Results	5
9	Honest Limits	5
10	What the Author Learned	6
11	Glossary	6

1 What and Why

raft-kv is a distributed key-value store in Python 3.12, with zero runtime dependencies, built around a from-scratch implementation of the **Raft consensus algorithm** (Ongaro & Ousterhout, USENIX ATC 2014, plus log compaction and single-server membership changes from Ongaro’s 2014 Stanford dissertation). **Consensus** means getting several independent nodes to agree on one ordered sequence of operations despite crashes, lost messages, and network partitions.

The project’s central architectural bet, called **sans-IO**: the class that implements Raft, **RaftNode**, never touches sockets, the wall clock, or Python’s global **random** – it only reacts to messages, timers,

and local calls, and only produces effects through two small injected interfaces, **Env** and **Storage**. That lets the identical class run either over a real asyncio TCP network or inside a deterministic, seeded simulator, so every bug the simulator finds is also a bug in the real production path.

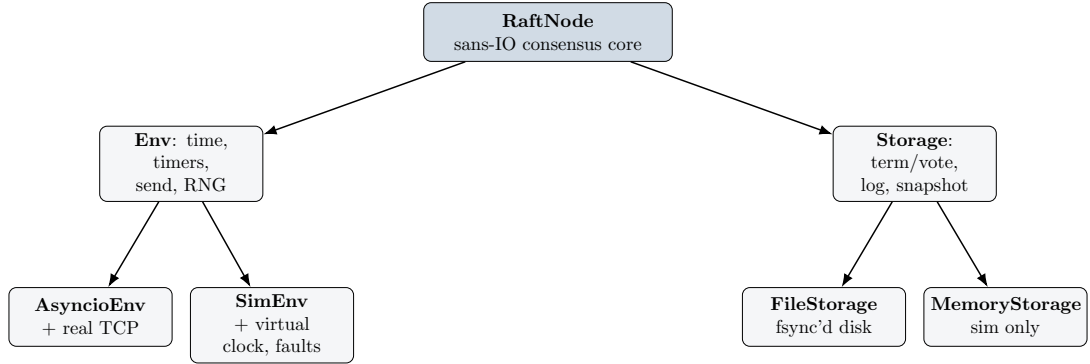


Figure 1: The sans-IO seam: one consensus core, two swappable worlds.

2 Project Layout

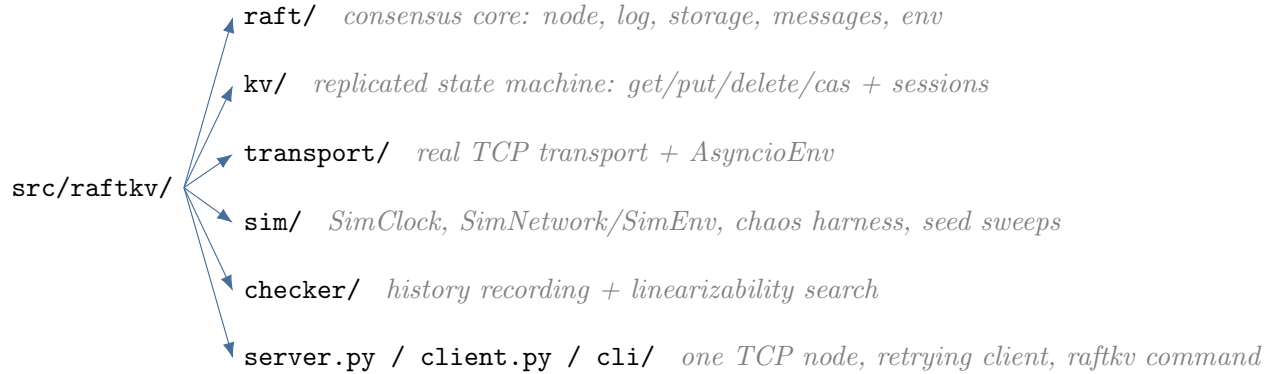


Figure 2: Roughly 3,700 lines across source and tests, organized by responsibility.

3 The Raft Core

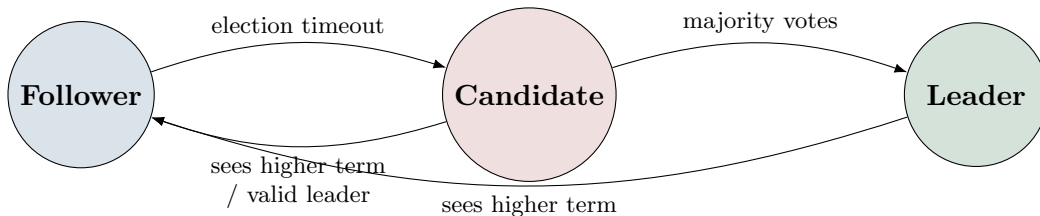


Figure 3: Raft's three roles and their transitions.

- **Elections:** randomized timeouts in $[T, 2T]$ avoid perpetual split votes; a candidate must win a majority (persisting its vote to disk before replying, so it never double-votes across a crash) and its log must be at least as up to date as the voter's (Section 5.4.1 of the paper).

- **Replication:** the leader tracks `next_index` / `match_index` per follower; on a log mismatch the follower replies with a **conflict index** so the leader can jump back many entries in one round trip (“fast backoff”) instead of retreating one entry at a time.
- **Commit rule:** the leader advances its commit index to the median of `match_index` (the highest index a majority holds), but – per the paper’s Section 5.4.2 – only counts entries from its *own current term* this way. A fresh leader appends a **no-op entry** in its own term so that older, already-agreed entries can commit through it.
- **Persistence:** `hardstate.json` (term/vote) is written atomically (temp file + fsync + rename); `log.bin` is an append-only, length- + CRC32-framed record file, so a crash mid-write leaves a detectable **torn tail** that is truncated on the next load. Nothing is ever acknowledged before its effect is fsync’d to disk – that single rule is what makes crash recovery safe.
- **Snapshots:** once enough entries accumulate past the last snapshot, the applied log prefix is replaced by a state-machine snapshot (KV data, sessions, and membership config); a follower too far behind receives the whole snapshot via `InstallSnapshot` (sent as one message, not chunked – fine at this project’s scale).
- **Membership changes** use the dissertation’s **single-server** approach (one add/remove per config entry, applied on *append* not commit) rather than joint consensus: any two configs differing by one member always share an overlapping majority, so split-brain during a transition is structurally impossible. Joint consensus (arbitrary jumps, two-phase) was judged unnecessary complexity for this project’s scope.

Linearizable reads: `ReadIndex`

A `get` never touches the log. The leader instead: (1) refuses to answer until an entry from its own term has committed (same reasoning as the no-op above); (2) captures `read_index = commit_index`; (3) runs one heartbeat round and waits for a majority’s acknowledgment, proving it is still recognized as leader *right now*; (4) waits for its local state to catch up to `read_index`, then serves the read from memory. A deposed leader stuck in a minority partition can never collect that majority ack, so it can never serve a stale read. The alternative, **leader leases** (serving reads for a bounded time after a heartbeat quorum, no per-read round trip), was deliberately not taken because its safety depends on an assumed bound on clock drift across machines – `ReadIndex` needs no such assumption, at the cost of one heartbeat round per read (shared across concurrent reads).

4 The Replicated State Machine

`KVMachine` supports `get/put/delete/cas` (compare-and-swap). Every write carries a (`client_id`, `seq`) pair; the machine remembers the highest `seq` applied per client and its result, so a retried write (the only safe response to an ambiguous timeout) returns the original cached result instead of re-executing. Because this session table rides inside every snapshot, it survives leader failover – upgrading the transport’s at-least-once delivery into **exactly-once** apply semantics. Cost: sessions never expire, an accepted memory leak at this project’s scale; a real deployment would expire them via a TTL agreed through the log.

5 Two Transports, One Determinism Story

Production: `TcpTransport` frames every message as a 4-byte length plus JSON; peer sends are fire-and-forget over persistent per-peer connections, matching Raft’s own tolerance for a lossy network.

Simulation: the whole point of `sim/` is to make an entire multi-node cluster a pure function of one integer seed:

- `SimClock` is a single event heap ordered by `(time, sequence)` – every timer, delivery, and fault is one event on it, so nothing ever really sleeps (12 virtual seconds of cluster time run in roughly 17ms of wall time).
- `SimNetwork` schedules each delivery with seeded random delay, drop probability, and duplication probability, and checks partitions *at delivery time* so an in-flight message during a forming partition is dropped, mirroring a real dying connection.
- `SimEnv` additionally models per-node clock skew (`timer_scale`) and simulated crashes (`kill()`, which invalidates every outstanding timer via an epoch counter).

One root seed fans out to per-network, per-cluster, per-nemesis, and per-client child RNGs, and a same-seed-twice fingerprint test guards against any accidental reliance on ambient (non-seeded) ordering.

6 The Chaos Harness

The harness entry point, `run_simulation`, takes one `SimConfig(seed=N)` and wires a 3-node `SimCluster`, a `Nemesis` that injects partitions/crashes/restarts/clock-skew at random times through the first 70% of a 12-second (virtual) run, and 3 concurrent `SimClients` issuing randomized get/put/cas with retries. A `Monitor` checks four safety invariants continuously, each failure naming its seed:

1. **Election safety** – at most one leader per term.
2. **Leader completeness** – a newly elected leader’s log must already contain everything ever applied cluster-wide.
3. **State machine safety** – no two nodes ever apply different content at the same log index.
4. **Log matching** – any two logs agreeing on (index, term) must hold byte-identical entries.

After the run, the recorded client history is checked for linearizability. `test_broken_raft_is_caught` deliberately weakens the commit rule and confirms the sweep catches the resulting split brain within a few dozen seeds – proof the safety net actually fires rather than merely existing.

7 The Linearizability Checker

A history is **linearizable** if every operation can be placed at some single instant between its invocation and its completion, in an order consistent with real time, such that a simple sequential model of the object reproduces every observed result. The checker implements the classical **Wing & Gong** exhaustive search with **Lowe’s memoization**, from scratch:

- Each key is checked independently (keys are independent registers) – the largest practical reduction in search space.
- Operations are bits in a bitmask, capping the search at 63 operations per key per history.

- At each step, any **minimal** operation (nothing un-linearized completed before its invocation) may be linearized next, if applying it to the model register reproduces the observed result; incomplete operations (no client response) branch both ways.
- Visited (`linearized-set`, `value`) states are memoized.
- Past a configurable state budget (2,000,000 by default), the checker returns an honest `UNKNOWN` – never a false `OK` or `VIOLATION`.

Checking linearizability is NP-complete in general; this is managed, not evaded. Known-bad synthetic histories (stale reads, lost updates, phantom values) in the test suite confirm the checker actually rejects violations – a checker that silently accepted everything would be worse than none at all.

8 Verification Results

Figures below are copied verbatim from the project’s README, run on the author’s own machine (Linux, Python 3.12.3):

- **pytest: 57 tests, all passing** – Figure 7 replication scenarios, torn-write recovery, membership changes over both transports, exactly-once failover retries, ReadIndex staleness, checker known-bad histories, the 200-seed slow sweep, and real 3-node TCP integration tests.
- The long randomized run:

```
$ raftkv sim --seeds 5000 --workers 10
5000 seeds in 84.7s (59.0 seeds/s), 564170 client ops total, 0 failure(s)
```

Each seed independently drove 12 virtual seconds of a 3-node cluster under partitions, crashes, restarts, 5% loss, 5% duplication, and clock skew, with every completed client history checked for linearizability.

9 Honest Limits

Directly from the README, not softened or expanded:

- Built and tested for single-digit node counts, localhost; throughput bounded by Python, JSON, and one `fsync` per append batch – a correctness project, not a fast store.
- Linearizability checker window: 63 ops/key max, honest `UNKNOWN` past its state budget.
- Snapshots ship whole, not chunked.
- No pre-vote: a healed, previously partitioned node forces one avoidable re-election before rejoining quietly.
- Client sessions never expire (unbounded dedup table growth).
- A newly added server only starts catching up after its membership-change entry is appended, briefly raising the effective commit quorum.

10 What the Author Learned

- Sans-IO is “the whole ballgame” for testable distributed systems: it turns “run 5000 chaos schedules” into an ordinary, fast function call, and makes every simulator-found bug a real one.
- Exactly-once is a state-machine feature, not a networking one – the dedup table must live inside the replicated state itself.
- Crash safety reduces to one discipline: never acknowledge before fsync, and make on-disk structures either atomic-rename or scan-and-truncate recoverable.
- Safety nets need their own tests, or they’re indistinguishable from safety nets that can never fire.
- Section 5.4.2’s commit restriction and the ReadIndex term-commit precondition are called out as the two places a plausible shortcut is actually a linearizability bug – and the chaos harness is what caught the second one.

Stated future work: pipelined replication with per-follower windows, chunked/cached snapshot transfer, pre-vote, injected torn writes in simulated storage, a persistent multiplexed client connection, and session expiry via a log-agreed TTL.

11 Glossary

Consensus

Getting independent nodes to agree on one ordered sequence of operations despite crashes and an unreliable network.

Quorum / majority

$\lfloor N/2 \rfloor + 1$ out of N nodes; any two majorities of the same set overlap by at least one node.

Linearizability

The guarantee that concurrent operations behave as if each took effect atomically at one instant, in an order consistent with real time.

fsync

Forces the OS to actually write buffered data to physical storage before returning, so it survives a crash.

Idempotent

Doing something twice has the same effect as doing it once (true of AppendEntries, which is why blind resending is safe).

Term

A monotonically increasing integer marking one election “epoch”; higher term always wins any comparison.